

Arduino Microcontroller Concepts (CSE 4820)



Program: B. TECH., 7th Semester

ITER, SIKSHA 'O' ANUSHANDHAN (Deemed to be) UNIVERSITY

Chapter 4:

Serial Communication

Learning Outcomes

- Explain the purpose and basic operation of serial communication in Arduino.
- Configure baud rate and use the Arduino Serial Monitor / Serial Plotter.
- Write sketches to transmit and receive characters and numeric data.
- Distinguish `Serial.print()`, `Serial.println()`, `Serial.write()`, `Serial.read()`, `Serial.available()`, and `Serial.peek()`.
- Design simple text-based message protocols using headers, fields, separators, and terminators.
- Understand hardware serial versus `SoftwareSerial` and identify voltage-level precautions.
- Debug a serial communication application using a systematic procedure.

4.0 What is Serial Communication?

- Serial communication transfers information sequentially, bit by bit, over a communication channel.
- In Arduino, it provides a simple way to communicate with a computer, another microcontroller, or a serial device.
- It is useful for data exchange, monitoring, debugging, configuration, and device control.
- The same USB serial connection used during sketch upload can also be used for application data.



Serial Communication: Basic Model

Transmitter side - Sends the data

- The **transmitter** is the device that wants to send information.
 - For example, Arduino has measured a temperature of **28°C**.

The transmitter performs these steps:

- Creates data or measurements
- Converts values to characters/bytes
- Places data into the serial transmit path
- Uses a defined baud rate and message format

Receiver side - Receives the data

- The **receiver** is the device that gets the transmitted information.
 - For example, the **computer** receives the value sent by Arduino.

It performs these steps:

- Checks whether data is available
- Reads characters/bytes from the receive buffer
- Interprets the message according to the agreed protocol
- Uses the received information to take an action

❑ Receive Buffer:

A receive buffer is a small temporary memory area where incoming serial data is stored until the program reads it.

Core Concepts of Serial I/O



Hardware UART

Universal Asynchronous Receiver-Transmitter handles physical bit timing via dedicated RX (Pin 0) and TX (Pin 1) lines connected to an internal 64-byte FIFO ring buffer.



Baud Rate Alignment

Communication speed measured in bits/second (e.g., 9600, 115200). Sender and receiver clocks must be identical to accurately sample asynchronous bit frames.



Framing & Protocols

Data is framed into 8 data bits, 1 stop bit, and no parity (8N1). Structured delimiters enable parsing formatted ASCII or packed binary streams.

4.1 Sending Information to Computer

Initializing Connection

Initialize serial hardware in `setup()` using `Serial.begin(baud)`. Standard speed is 9600 bps, but 115200 is preferred for high-throughput sensor telemetry.

```
void setup() { Serial.begin(9600); // Start UART }
```

Printing Text & Data

`Serial.print()` transmits ASCII strings without terminating characters. `Serial.println()` automatically appends Carriage Return (`'\r'`) and Linefeed (`'\n'`).

```
void loop() { Serial.print("Sensor: ");  
Serial.println(analogRead(A0)); }
```

Arduino Serial Monitor and Serial Plotter

Serial Monitor

- Displays text and numeric data received from Arduino.
- Can also send text/characters from the computer to Arduino.
- Baud rate must match `Serial.begin()`.
- Line-ending options include No line ending, Newline, Carriage return, and Both NL & CR.

Serial Plotter

- Provides a graphical view of serial data.
- Useful for observing changing sensor values or trends.
- Close the Serial Monitor before opening the Plotter.
- Useful as a quick visualization/debugging tool.

Sending Information to the Computer

Initialize the serial port in setup().

Use Serial.print() for text/data without automatically starting a new line.

Use Serial.println() to print and then move to a new line.

The book's example sends an increasing number every 500 ms.

```
void setup() {  
    Serial.begin(9600);  
}  
  
int number = 0;  
  
void loop() {  
    Serial.print("The number is ");  
    Serial.println(number);  
    delay(500);  
    number++;  
}
```

print(), println(), and write()

- Serial.print(value) sends a human-readable representation of a value.
- Serial.println(value) performs the print and then adds a line ending.
- Serial.write(value) is intended for sending raw bytes/characters rather than formatted human-readable output.
- print() supports formats such as DEC, HEX, OCT, and BIN for integer values.

Choose text when readability and debugging are important; binary can be more efficient but is harder to inspect.

```
Serial.print(65);           // decimal text
Serial.print(65, HEX);      // hexadecimal text
Serial.print(65, BIN);      // binary text
Serial.write(65);           // raw byte
```

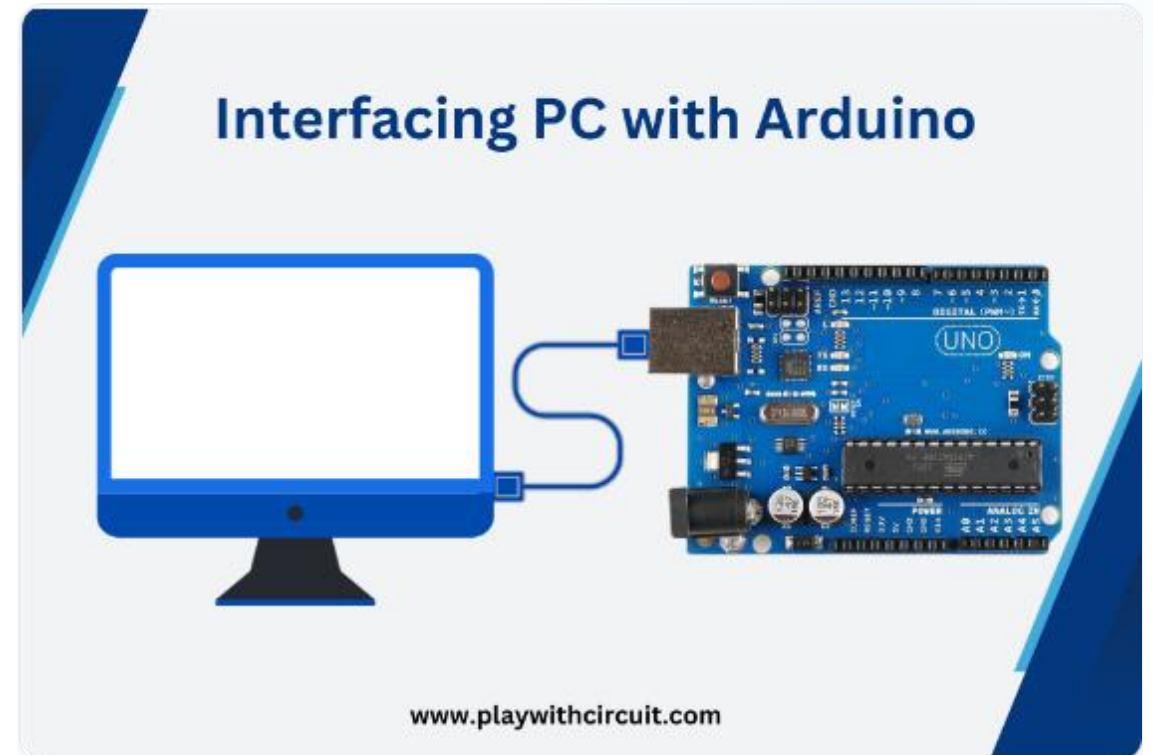
4.3 Receiving Serial Data

Receiving bytes requires checking buffer status before performing read operations to prevent blocking executions:

```
void loop() { if (Serial.available() > 0) { char ch =  
Serial.read(); if (ch == 'H') { digitalWrite(13, HIGH); } else  
if (ch == 'L') { digitalWrite(13, LOW); } } }
```

Serial.available()

returns unread byte count in the 64-byte RX hardware buffer.



Important Serial Functions — Quick Reference

Function	Purpose
Serial.begin(baud)	Initialize serial communication
Serial.print()	Send formatted text/data
Serial.println()	Send data + line ending
Serial.write()	Send raw byte/character data
Serial.available()	Check received data in buffer
Serial.read()	Read next received byte/character
Serial.peek()	Inspect next byte without removing it
Serial.flush()	Wait for outgoing transmission to finish

Sending Multiple Text Fields

Combining several sensor values in one serial message

Problem: A project must send more than one value at a time.

Solution: Send fields in one text line separated by a delimiter such as comma (,).

Example message: 25.4,61.2,512

The receiver splits the message using the known delimiter.

Advantages: human-readable, easy to debug, compatible with Serial Monitor and Processing.

```
Serial.print(temperature); Serial.print(",");  
Serial.print(humidity); Serial.print(",");  
Serial.println(lightLevel);
```

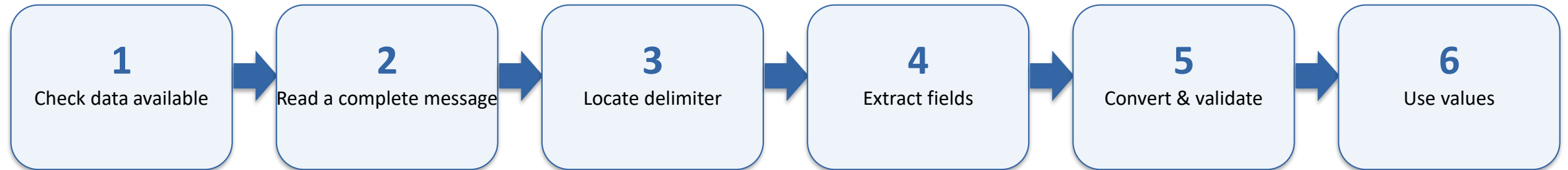
4.5 Receiving Multiple Text Fields

Arduino receives a complete line containing several fields.

- Read characters until the end-of-line marker is detected.
- Separate fields using the delimiter.
- Convert numeric text to numbers before calculations.
- Typical tools: `Serial.available()`, `Serial.read()`, String functions, `parseInt()`/`parseFloat()`.

```
if (Serial.available()) {  
    String msg = Serial.readStringUntil('\n');  
    int comma = msg.indexOf(',');  
    // extract fields and convert as required  
}
```

Receiving Algorithm



4.6 Sending Binary Data

Transmit raw byte values instead of ASCII text

Text vs Binary

Text: number 100 may require 3 ASCII bytes: '1','0','0'.

Binary: value 100 can be represented directly as one byte.

Binary can reduce message size and increase speed.

Binary is less human-readable.

Arduino Tools

`Serial.write(byte)`

`Serial.write(buffer, length)`

Use fixed data types: `byte`, `int`, `uint16_t`, `float`, etc.

Define packet layout before communication begins.

Binary Transmission Example

```
byte value = 100;  
Serial.write(value);           // sends one raw byte  
  
byte packet[3] = {10, 20, 30};  
Serial.write(packet, 3);      // sends 3 bytes
```

`Serial.print(100)` → sends characters representing 100.

`Serial.write(100)` → sends the byte value 100.

Always make sure the receiver interprets the same data format.

4.7 Receiving Binary Data on a Computer

A PC application can read the serial port as a stream of bytes.

The application must know the packet format and byte order.

Binary packets are useful for high-rate sensor acquisition.

Common environments: Processing, Python, MATLAB, LabVIEW and custom serial applications.

Include packet markers, length fields or checksums when reliability matters.

```
Arduino → USB/Serial → PC application  
bytes: [HEADER][LENGTH][DATA][CHECKSUM]
```

4.8 Sending Binary Values from Processing to Arduino

Processing can act as a serial controller for Arduino.

Arduino receives bytes and interprets them according to the agreed protocol.

Useful for GUI control, interactive visualization and PC-based experiments.

Both sides must agree on baud rate, packet format and data type.

```
// Processing concept
myPort.write(120);      // send one byte

// Arduino concept
if (Serial.available()) {
  int value = Serial.read();
}
```

4.5 Stream Parsing Methods

What is stream parsing?

A serial port receives data as a continuous stream of characters.

For example, suppose the PC sends:

25,60

Arduino receives the characters: '2' → '5' → ',' → '6' → '0'

Arduino needs to find the numbers inside this stream. This process is called parsing.

Stream.parseInt()

Scans incoming ASCII stream for integer digits. Non-digit characters are automatically skipped until a numerical sequence is parsed.

```
int v1 = Serial.parseInt(); int v2 = Serial.parseInt();  
if (Serial.read() == '\n') { // Process integer pair }
```

Stream.parseFloat()

Parses floating-point representations from the serial buffer. Reads signs, integer components, decimal points, and fractional values.

```
float temp = Serial.parseFloat(); float hum =  
Serial.parseFloat(); // Automatically handles  
comma/space separators
```

4.6 Sending Binary Data

ASCII vs Binary Bandwidth

ASCII formatting turns number 12345 into 5 distinct byte characters ('1','2','3','4','5'). Binary transfer sends raw bits directly as a 2-byte integer, cutting bandwidth usage by 60%.

Serial.write() Mechanics

Serial.write() transfers raw binary bytes directly to the UART buffer. Crucial for high-frequency sensor telemetry, motor drivers, and DSP systems.

```
byte val = 123; Serial.write(val); // Sends 1 raw byte
int num = 1000; Serial.write(lowByte(num)); // LSB
Serial.write(highByte(num)); // MSB
```

4.9 Sending Values of Multiple Arduino Pins

Read several digital/analog inputs and send their values as one record.

Useful for monitoring multiple sensors simultaneously.

Example: digital pins 2–5 + analog pins A0–A2.

A fixed field order makes the data easy to parse and plot.

```
for (int pin = 2; pin <= 5; pin++) {  
    Serial.print(digitalRead(pin));  
    Serial.print(",");  
}  
Serial.println(analogRead(A0));
```

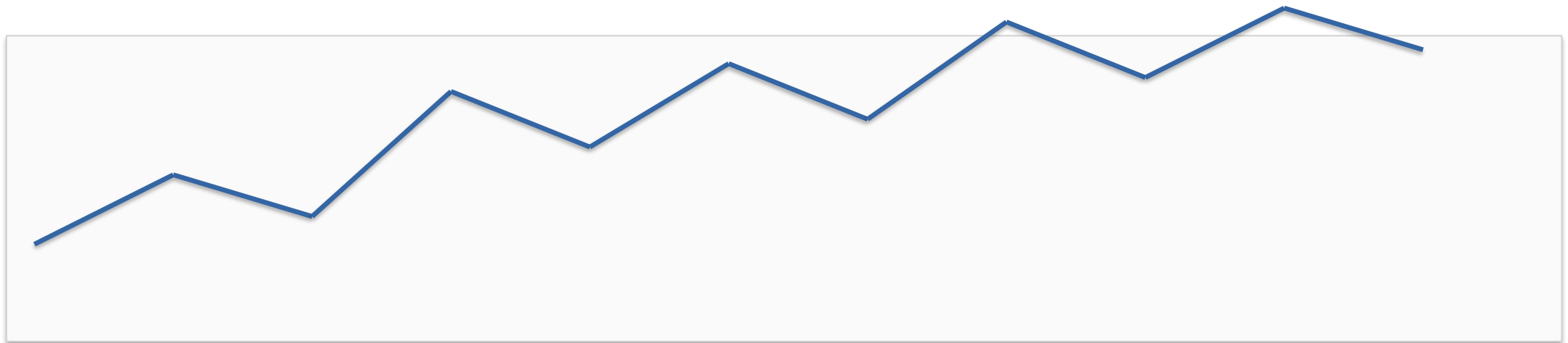
Serial Plotter for Multi-Channel Data

Arduino IDE Serial Plotter provides a quick visual view of numeric serial data.

Send multiple numeric values on one line to compare channels.

Example: sensor1, sensor2, sensor3.

Applications: vibration, temperature, pressure, TENG output and sensor testing.



4.10 Logging Arduino Data to a File

Typical Workflow

Arduino measures sensor data.
Serial sends timestamp + values.
PC application reads incoming lines.
Data are appended to a CSV/text file.
File can be analyzed later in MATLAB/Excel/Python.

Recommended Record

time_ms,temperature,humidity,voltage
Keep a consistent column order.
Record sampling interval and units.
Check for missing/corrupted records.

Data Logging Example

```
// Arduino sends CSV-style records  
Serial.print(millis());  
Serial.print(",");  
Serial.print(sensorValue);  
Serial.print(",");  
Serial.println(voltage, 3);
```

Example output: 1520,487,2.315

CSV fields can be imported into MATLAB, Excel or Python.

For experiments, include units, calibration information and sampling rate in the project documentation.

4.11 Sending Data to More Than One Serial Device

Some Arduino boards provide multiple hardware serial ports.

Use Serial, Serial1, Serial2, etc., where supported by the board.

A common arrangement: Serial/USB → computer; Serial1 → external device.

Each port can be initialized with its own baud rate.

On boards with only one hardware UART, a software serial library may be used for an additional device.

```
void setup() {  
  Serial.begin(9600);    // PC/USB  
  Serial1.begin(9600);   // external serial device  
}
```

Hardware Serial vs Software Serial

Hardware Serial

Uses dedicated UART hardware.

More reliable and efficient.

Better for higher data rates.

Board dependent: Uno has one main hardware UART;
Mega has multiple.

Software Serial

Emulates serial communication using GPIO pins.

Useful when extra serial ports are needed.

Consumes processor time.

Best for moderate/low data rates.

4.12 Receiving Serial Data from More Than One Device

Example: Arduino receives GPS data while maintaining a PC serial connection.

Boards with multiple UARTs can dedicate one port to each device.

With one hardware UART, SoftwareSerial can create an additional port.

Choose the hardware port for the device with greater bandwidth whenever possible.

Always verify voltage levels and common ground before connecting serial devices.

```
Serial.begin(9600);      // computer
serial_gps.begin(9600);  // GPS / external device

if (serial_gps.available()) {
  char c = serial_gps.read();
  Serial.write(c);        // echo to PC
}
```

Questions & Answers

Q1. Why should `Serial.available()` be checked before using `Serial.read()`?

Answer:

`Serial.available()` checks whether data is present in the receive buffer. It prevents the program from attempting to read nonexistent data and allows the Arduino to process incoming characters only when they are available.

Q2. An Arduino sends data at 9600 baud, but the Serial Monitor is configured for 115200 baud. What will happen and why?

Answer:

The received information may appear as **garbled or unreadable characters** because both ends are using different communication settings. The baud rate must be matched at the transmitter and receiver.

Q3. Why is `Serial.println()` generally more convenient than `Serial.print()` when transmitting multiple data values?

Answer:

`Serial.println()` automatically adds a **line ending**, making each transmission appear on a separate line. This is particularly useful for displaying successive measurements and for Serial Plotter visualization.

Q4. An Arduino receives the character '7'. Explain how it can be converted into the numerical value 7.

Answer:

The received value is a character, not the number 7. It can be converted using:

```
int value = ch - '0';
```

Since ASCII digit characters are sequential, subtracting '0' gives the corresponding numerical value.

Q5. Why is a header useful in a serial communication message such as H,25,60?

Answer:

The header H identifies the **type or beginning of the message**. It allows the receiving Arduino to distinguish the intended message from other incoming characters or data.

Q6. Consider the message T,25,60\n. What are the roles of T, commas, and \n?

Answer:

- T → **header/message identifier**
- , → **field separator**
- \n → **message terminator**

This structure allows the receiver to identify and separate the transmitted information.

Q7. Why is a structured serial protocol preferred when transmitting several sensor values?

Answer:

A structured protocol provides a defined format for identifying and separating multiple values. It makes the received data **easier to parse, interpret, validate, and process**.

Q8. What is the advantage of using Serial.write() instead of Serial.print() for raw data transmission?

Answer:

Serial.write() transmits **raw byte data**, whereas Serial.print() converts values into human-readable characters. Therefore, Serial.write() is more appropriate when the receiving device expects a specific byte-level format.

Q9. An Arduino Uno communicates with an RS-232 device. Can its TX pin be connected directly to the RS-232 input? Justify.

Answer:

No. Arduino uses logic-level serial signals, whereas RS-232 uses different electrical voltage levels. An appropriate **RS-232 interface/level converter** must be used.

Q10. Why might SoftwareSerial be selected instead of the Arduino's hardware serial port?

Answer:

SoftwareSerial can provide serial communication on selected digital pins when the **hardware serial port is already being used**, for example, for communication with the computer.

Q11. Why is hardware serial generally preferred over SoftwareSerial when available?

Answer:

Hardware serial uses the microcontroller's dedicated serial hardware, providing **better timing and performance** than software-emulated serial communication, particularly when communication speed increases.

Q12. A potentiometer is connected to A0. How can its changing value be observed using the Serial Plotter?

Answer:

The analog value can be read using `analogRead(A0)` and transmitted using `Serial.println()`:

```
int value = analogRead(A0);
```

```
Serial.println(value);
```

The Serial Plotter then displays the changing value graphically.

Q13. Why is Serial Plotter more suitable than Serial Monitor for observing a changing sensor signal?

Answer:

Serial Monitor displays numerical values as text, whereas Serial Plotter represents numerical data **graphically with respect to successive samples**. Therefore, trends and variations in sensor output are easier to observe.

Q14. A serial message contains H,100,250,75. Why might Serial.parseInt() be useful for processing this message?

Answer:

Serial.parseInt() can extract the numerical fields from the incoming serial stream. Thus, values such as **100, 250, and 75** can be obtained without manually converting each digit character.

Q15. A serial-controlled Arduino system sometimes interprets incomplete commands. Suggest two improvements to the communication protocol.

Answer:

Two improvements are:

Use a **clear header and message terminator** to identify complete messages.

Define and validate the **expected number and format of fields** before processing the command.

This makes the communication more reliable and reduces incorrect interpretation of incomplete data.

Code-Based 2-Mark Questions

Q1. What will be the output of the following code?

```
Serial.begin(9600);  
Serial.println("Arduino");  
Serial.println(25);
```

Answer:

The Serial Monitor displays:

Arduino

25

Q2. What is the function of the following statement?

```
char ch = Serial.read();
```

Answer:

It reads the **next available byte/character** from the serial receive buffer and stores it in the variable ch.

Q3. What is the purpose of the following statement?

```
if (Serial.available())
```

Answer:

It checks whether **serial data is available in the receive buffer** before attempting to read it.

Q4. What does the following code do?

```
if (ch == '1')  
    digitalWrite(LED_BUILTIN, HIGH);
```

Answer:

If the received character is '1', the Arduino switches the **built-in LED ON**.

Q5. What is the difference between the following statements?

```
Serial.print(65);
```

```
Serial.write(65);
```

Answer:

`Serial.print(65)` sends the characters representing **65**, whereas `Serial.write(65)` sends the **raw byte value 65**.